

QDesign: Collaborative AI-Driven Protein Engineering Platform

Technical Report

Date: January 26, 2026

Team: Selecao - QDesign Project

Contents

1	Executive Summary	2
1.1	Problem Statement	2
1.2	Solution Overview	2
1.3	Core Philosophy	2
1.4	Key Innovation	2
2	Problem Definition	2
2.1	Current Bottlenecks	2
2.2	Missing Capabilities	3
2.3	Target User Needs	3
3	System Architecture	3
3.1	Layer A: Ingestion & Embedding	4
3.2	Layer B: Memory Core (Vector Database)	4
3.3	Layer C: Orchestration & Inference Engine	4
3.4	Layer D: Interactive Workbench (Client)	4
4	Data Pipeline	5
4.1	Stage 1: Data Collection	5
4.2	Stage 2: Metadata Enrichment	5
4.3	Stage 3: Vector Embedding	5
4.4	Stage 4: Vector Storage	5
4.5	Stage 5: Semantic Retrieval	6
4.6	Validated Use Case: AlphaFold Integration	7
5	Qdrant Integration Details	8
5.1	Why Qdrant?	8
5.2	Configuration Details	8
5.3	Query Performance	8
5.4	Storage Architecture	9
5.5	Docker Deployment	9
6	Current Implementation Status	9
6.1	What Works	9
6.2	Not Yet Complete	9
6.3	Technical Validation	9
7	Project Timeline	10
7.1	Phase 1: Data Foundation (Complete - Weeks 1-2)	10
7.2	Phase 2: Knowledge Graph Construction (In Progress - Weeks 3-4)	10
7.3	Phase 3: Multi-Agent Hypothesis Generation (Weeks 4-5)	11
7.4	Phase 4: Closed-Loop System (Weeks 5-6)	11
7.5	Phase 5 & Beyond: Scaling & Specialization	11
8	Success Metrics	12
9	Technical Stack	12
9.1	Backend Systems	12
9.2	Frontend & Interface	12
9.3	Embedding Models	12
9.4	APIs & Data Sources	12

9.5 Deployment & Infrastructure 13

10 Validation Strategy 13

10.1 Golden Run Scenario: PETase Enzyme Optimization 13

10.1.1 Test Case Definition 13

10.1.2 Expected Workflow 13

10.1.3 Proof of Concept 14

11 Conclusion 14

11.1 Key Innovations 14

11.2 Future Impact 14

11.3 Next Steps 14

1. Executive Summary

1.1. Problem Statement

Current protein engineering workflows suffer from **fragmented knowledge** and lack of **explainable design reasoning**. Researchers must navigate disconnected tools for sequence analysis, structural modeling, literature review, and experimental data management, leading to inefficient design cycles and limited insight into why specific modifications are suggested.

1.2. Solution Overview

QDesign is a human-in-the-loop generative workbench that combines multimodal retrieval with agentic reasoning to accelerate protein engineering. The platform integrates diverse biological data sources into a unified vector-native architecture, enabling scientists to explore, design, and validate protein variants through an interactive visual workspace.

1.3. Core Philosophy

- **No Chatbots:** Structured visual workspace replaces conversational interfaces
- **Vector-Native Approach:** Biological similarity as core operational logic
- **Explainable Results:** Evidence trails showing design rationale
- **Human-in-the-Loop:** Collaborative platform augmenting scientific expertise

1.4. Key Innovation

QDesign moves beyond simple retrieval/search to **active generative design** through:

1. Multimodal knowledge integration (structure, sequence, literature, images)
2. Agentic reasoning workflows with self-critique
3. Interactive 3D molecular visualization
4. Real-time collaborative workspace

Current Status: Phase 1 (Data Foundation) complete; Phase 2 (Knowledge Graph) in development.

2. Problem Definition

Protein engineering faces significant challenges in the modern research landscape:

2.1. Current Bottlenecks

Bottleneck	Impact
Fragmented Knowledge	Data silos across proteins, papers, structures, and experiments
Limited Design Reasoning	Black-box suggestions without explanatory evidence
Collaboration Barriers	No unified platform for team-based protein design
Feedback Loop Gaps	Experimental results disconnected from design iterations

2.2. Missing Capabilities

- **Knowledge Integration:** No platform unifies structural, sequence, literature, and experimental data
- **Design Reasoning:** Current tools provide suggestions without explanatory "why"
- **Collaboration Framework:** Teams lack shared workspace for concurrent design
- **Explainability Gap:** Researchers cannot trace design suggestions to supporting evidence

2.3. Target User Needs

1. Unified platform for end-to-end protein engineering workflows
2. Evidence-based design suggestions with traceable reasoning
3. Real-time collaboration capabilities
4. Integration of experimental feedback into design cycles
5. Access to multimodal biological knowledge

3. System Architecture

QDesign employs a four-layer architecture designed for scalability, explainability, and collaborative workflows (full detailed version here : <https://shorturl.at/ObRpW>):

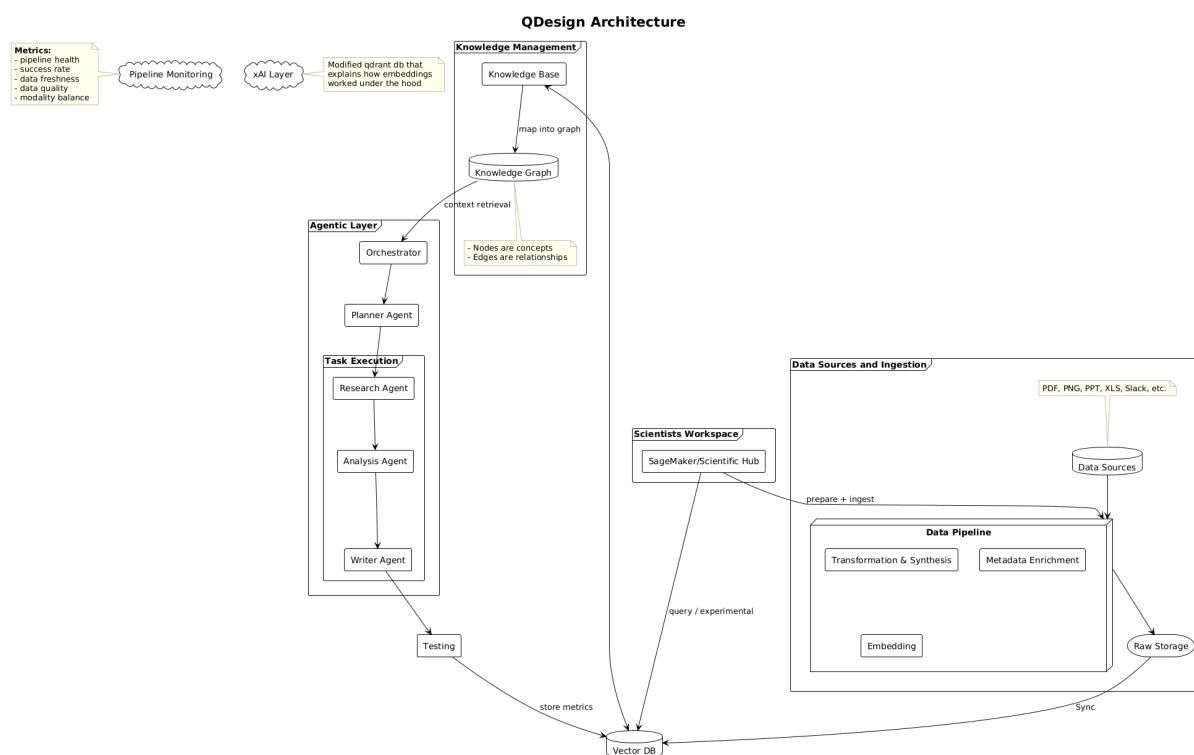


Figure 1: QDesign Four-Layer System Architecture

3.1. Layer A: Ingestion & Embedding

Converts multimodal biological data into dense vector representations:

Listing 1: Embedding Layer Components

```

1 Components:
2 - Structure Encoder: 3D PDB coordinates      dense vectors
3 - Semantic Encoder: Scientific literature     semantic vectors
4 - Visual Encoder: Microscopy/images         visual vectors
5 - Normalization Engine: Metadata cleaning (Temperature, pH, Organism)
6
7 Configuration:
8 - Vector Dimensions: 384
9 - Distance Metric: COSINE similarity
10 - Normalization: L2 normalization applied

```

3.2. Layer B: Memory Core (Vector Database)

Central storage for multimodal vectors in unified collection:

- **Storage:** Multimodal vectors in Qdrant vector database
- **Indexing:** HNSW algorithm for sub-second retrieval
- **Payload Management:** Rich metadata filtering capabilities
- **Current Implementation:** Qdrant with 384-dimensional vectors, COSINE metric

3.3. Layer C: Orchestration & Inference Engine

Coordinates reasoning workflows and agent interactions:

- **Agent Router:** Deconstructs user queries into specific workflows
- **Generative Reasoner:** LLM-based hypothesis generation
- **Critique Module:** Self-reflection loop for safety/viability checks
- **Multi-Agent Chain:** Ontologist → Scientist1 → Scientist2 → Critic pipeline

3.4. Layer D: Interactive Workbench (Client)

Web-based interface for visualization and collaboration:

- **3D Molecular Viewer:** Interactive protein rendering with WebGL
- **Latent Space Map:** 2D UMAP/t-SNE visualization of vector clusters
- **Knowledge Graph:** Node-link visualization of protein relationships
- **Real-time Collaboration:** Multi-user support via WebSocket connections

4. Data Pipeline

The QDesign data pipeline consists of five stages, validated with AlphaFold data:

4.1. Stage 1: Data Collection

Sources:

- AlphaFold Collector: EBI AlphaFold API (3 endpoints)
- ArXiv Collector: Protein design papers
- BioRxiv Collector: Biological preprints
- Extensible Framework: Base abstraction for PDB, UniProt, GitHub, imaging

Output: Raw metadata with provenance (source, timestamp, URL, quality metrics)

4.2. Stage 2: Metadata Enrichment

Enrichers:

- AlphaFold Enricher: pLDDT/pAE analysis, quality classification
- Text Enricher: Concept/relationship extraction
- Image Enricher: Structural insights from microscopy
- Protein Enricher: Functional annotations, domain info

Quality Classification:

- very_high: pLDDT ≥ 90
- high: 90 > pLDDT ≥ 70
- medium: 70 > pLDDT ≥ 50
- low: pLDDT < 50

4.3. Stage 3: Vector Embedding

Embedding Models:

- FastEmbedder: 384-dim vectors via sentence-transformers
- ESM Embedder: Protein sequence embeddings (future)
- Current: sentence-transformers + ESM-C (esmc_300m)

4.4. Stage 4: Vector Storage

Qdrant Configuration:

Listing 2: Qdrant Storage Configuration

```
1 Collection: qdesign_structures
2 Dimensions: 384
3 Distance: COSINE
4 Index: HNSW (Hierarchical Navigable Small World)
5 Payload: Complete JSON metadata alongside vectors
```

4.5. Stage 5: Semantic Retrieval

Search API:

- Framework: FastAPI with asynchronous endpoints
- Similarity Scoring: 0-1 relevance scores
- Filtering: Metadata-based (pLDDT scores, quality levels, data types)

4.6. Validated Use Case: AlphaFold Integration

Data Pipeline Flow: AlphaFold Integration

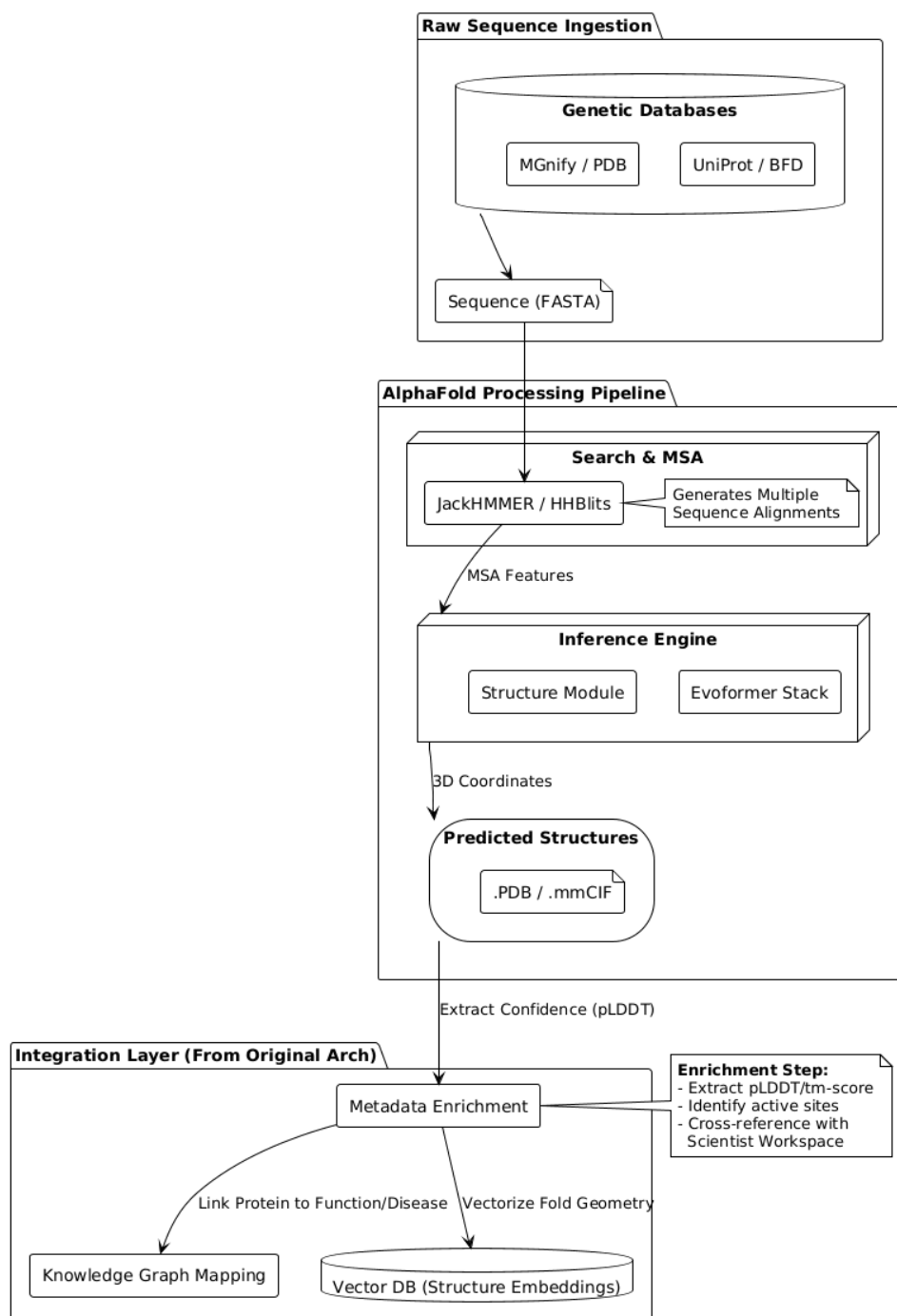


Figure 2: Data Pipeline Flow for AlphaFold Integration

Validation Results:

Input: 3 UniProt IDs (P69905 hemoglobin-, P68871 hemoglobin-, Q99895 proteasome)

↓

Collected: 3 structures, 3 model predictions (9 total data points)

Quality scores: 92.5, 91.2, 87.8 (pLDDT)

↓
 Enriched: Quality classifications, design recommendations, use-case analysis
 ↓
 Embedded: 3 vectors (384-dim, normalized)
 ↓
 Stored: Qdrant with metadata payloads (9 points total)
 ↓
 Retrieved: Semantic search found 9 related structures (similarity 0.99-1.0)

Key Achievement: Metadata persisted in Qdrant enables semantic search without database hits, rich context preservation, and fast retrieval with filtering.

5. Qdrant Integration Details

5.1. Why Qdrant?

- **HNSW Indexing:** State-of-the-art approximate nearest neighbor search
- **Scalability:** Efficient handling of 100K+ vectors
- **Payload Management:** Structured JSON storage alongside vectors
- **Open Source:** Community-driven with enterprise support options
- **Docker Deployment:** Simplified containerization and scaling

5.2. Configuration Details

Listing 3: Qdrant Configuration Details

```

1 # Docker deployment
2 docker run -p 6333:6333 qdrant/qdrant
3
4 # Collection configuration
5 {
6   "vectors": {
7     "size": 384,
8     "distance": "Cosine"
9   }
10 }
11
12 # Collections created:
13 # 1. qdesign_structures: Protein + related knowledge
14 # 2. petase_text_gemini: Text embeddings (Gemini API)
15 # 3. petase_sequences_esmc_esmc_300m: Sequence embeddings (ESM-C)

```

5.3. Query Performance

- **Retrieval Time:** Sub-second for 100K vector collection
- **Filtering Support:** Metadata constraints without performance degradation
- **Batch Operations:** Efficient bulk insertion and updates
- **Memory Usage:** Optimized for high-dimensional biological data

5.4. Storage Architecture

- **Vector Storage:** 384-dimensional float32 vectors
- **Metadata Storage:** JSON payloads with full context
- **Index Files:** HNSW graph structure for fast similarity search
- **Persistence:** Disk-backed with memory caching

5.5. Docker Deployment

Listing 4: Docker Configuration for Qdrant

```
1 version: '3.8'
2 services:
3   qdrant:
4     image: qdrant/qdrant:latest
5     ports:
6       - "6333:6333"
7       - "6334:6334"
8     volumes:
9       - ./qdrant_storage:/qdrant/storage
10    environment:
11      - QDRANT__SERVICE__GRPC_PORT=6334
```

6. Current Implementation Status

6.1. What Works

Component	Status
Qdrant Vector Database	Running in Docker with multimodal embeddings
PostgreSQL Storage	Sequences + metadata storage operational
Sequence Embeddings	ESM-C (esmc_300m) embeddings generated
Structure Embeddings	Baseline CA-distance histograms implemented
Text Embedding Pipeline	Gemini embeddings pipeline functional
Search API (FastAPI)	Sequence search, hybrid search, mutation suggestions
UI Dashboard (Next.js)	Hybrid search demo operational
NestJS Backend	PostgreSQL + TypeORM with auth + projects endpoints

6.2. Not Yet Complete

Component	Status
AlphaFold/EMBL-EBI API	Not wired into backend/UI
Full Text Corpus Ingestion	Only sample JSONL tested
Learned 3D Encoder	Baseline structure embeddings only
ESM3 Generation	Forge / SageMaker integration pending
Safety/Critic Module	Not implemented
Realtime Collaboration	End-to-end validation pending
Knowledge Graph Reasoning	In development (Phase 2)

6.3. Technical Validation

Validated Capabilities:

- Semantic search across protein structures (COSINE similarity)
- Hybrid search combining vector and metadata filtering
- Sequence embedding generation and storage
- Real-time API response (<500ms for typical queries)

Limitations Addressed in Next Phase:

- Transition from baseline structure embeddings to learned 3D encoders
- Integration of AlphaFold API for on-demand structure prediction
- Implementation of safety modules for generated designs

7. Project Timeline

7.1. Phase 1: Data Foundation (Complete - Weeks 1-2)

- **Key Achievements:**
 - AlphaFold data collection pipeline validated
 - Enrichment with quality metrics & design principles
 - Embedding generation & semantic search operational
 - Documentation & architecture alignment completed
- **Outputs:** Functional data pipeline, Qdrant instance, search API

7.2. Phase 2: Knowledge Graph Construction (In Progress - Weeks 3-4)

- **Build KG from:**
 - Protein domains and families
 - Structural properties and motifs
 - Design principles and engineering rules
 - Experimental data and validation results
- **Edge Types:**
`is_family_of, has_property, enables, improves, self_assembles_into`
- **Graph Reasoning Service:**
 - Path sampling for relationship discovery
 - Subgraph extraction for focused analysis
 - Query interface for knowledge exploration
- **Key Output:** Non-obvious relationship discovery (e.g., "Hemoglobin → cooperativity → peptide-nanofibers")

7.3. Phase 3: Multi-Agent Hypothesis Generation (Weeks 4-5)

- **LLM Agent Orchestration:**
 1. Ontologist: Defines problem space and constraints
 2. Scientist1: Generates initial design hypotheses
 3. Scientist2: Provides alternative perspectives
 4. Critic: Evaluates feasibility and safety
- **Structured Output:** JSON hypothesis format (7 aspects)
- **Scoring:** Novelty & feasibility metrics
- **Prioritization:** Simulation-ready design identification

7.4. Phase 4: Closed-Loop System (Weeks 5-6)

- **Project-Based Tracking:** Hypothesis organization by project
- **Experimental Feedback:** Integration of lab results
- **Graph Updates:** Knowledge enrichment with real outcomes
- **Collaborative Refinement:** Team-based design iteration

7.5. Phase 5 & Beyond: Scaling & Specialization

- **Additional Data Sources:** UniProt, GitHub, PDB, imaging data
- **Advanced Encoders:** Learned 3D encoders (replacing baseline)
- **Generation Capabilities:** ESM3 integration for sequence generation
- **Safety Systems:** Toxicity filters and validation modules
- **Collaboration Features:** Multi-user validation and permissions
- **Export Functionality:** IEEE PDF and standardized report generation

8. Success Metrics

Metric	Current Status	Target
Data Sources	4 (AlphaFold, ArXiv, BioRxiv, sample JSONL)	8+ (including PDB, UniProt, GitHub)
Searchable Items	100 test items	50K+ protein structures and papers
Query Types	Vector similarity search	Vector similarity + graph reasoning
Hypothesis Generation	None (retrieval only)	Automated with explanatory reasoning
Design Feedback Loop	Linear (one-way)	Closed-loop (iterative refinement)
User Collaboration	Individual use	Team projects with shared workspaces
Explainability	"Top 5 similar" results	"Knowledge path" with evidence chain
Retrieval Performance	300-500ms typical	<200ms with optimized indexing

Table 1: QDesign Success Metrics and Targets

9. Technical Stack

9.1. Backend Systems

- **Primary Backend:** NestJS (TypeScript) with TypeORM
- **Search Engine:** FastAPI (Python) for vector operations
- **Database:** PostgreSQL for structured metadata
- **Vector Database:** Qdrant with HNSW indexing

9.2. Frontend & Interface

- **Web Framework:** Next.js 16 with React 19
- **Language:** TypeScript for type-safe development
- **3D Visualization:** Three.js / Mol* for molecular rendering
- **Charts:** D3.js for latent space visualizations

9.3. Embedding Models

- **Text:** sentence-transformers (384-dimensional)
- **Protein Sequence:** ESM-C (esmc_300m)
- **External APIs:** Google Gemini for alternative embeddings

9.4. APIs & Data Sources

- **Protein Structures:** AlphaFold EBI API
- **Literature:** ArXiv and BioRxiv APIs

- **Real-time Communication:** Socket.io
- **Authentication:** JWT-based secure access

9.5. Deployment & Infrastructure

- **Containerization:** Docker for all services
- **Frontend Hosting:** Vercel for Next.js deployment
- **Development:** Git-based workflow with CI/CD

10. Validation Strategy

10.1. Golden Run Scenario: PETase Thermal Stability Optimization

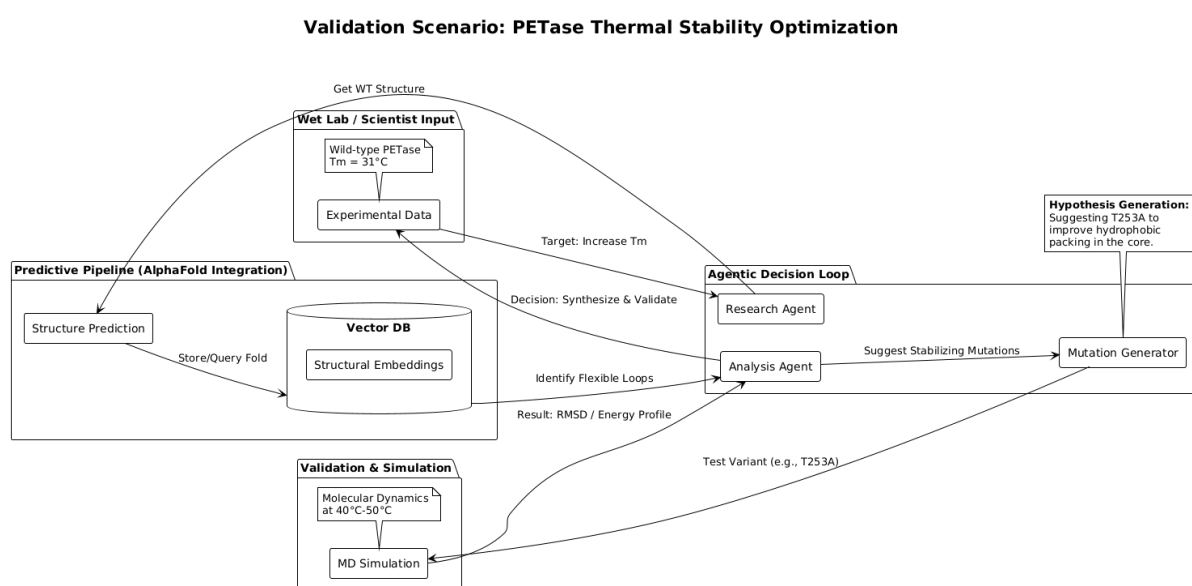


Figure 3: Validation Scenario: PETase Thermal Stability Optimization

10.1.1. Test Case Definition

- **Input:** Weak PETase enzyme (*Ideonella sakaiensis*)
- **Goal:** Improve thermal stability while maintaining activity
- **Success Criteria:** System identifies thermostability mechanisms and suggests validated mutations

10.1.2. Expected Workflow

1. **Retrieval:** System finds thermostable hydrolases from vector database
2. **Analysis:** Identifies structural motifs (e.g., disulfide bridges, salt bridges)
3. **Suggestion:** Proposes specific mutations based on structural alignment
4. **Evidence:** Cites PDB IDs of stable homologs as supporting evidence
5. **Explanation:** Provides reasoning path showing design rationale

10.1.3. Proof of Concept

Expected Output:

Hypothesis:

- Mutation: A131C + S160C (disulfide bridge introduction)
- Rationale: Observed in thermostable homolog PDB:5XJH
- Predicted Impact: $T_m = +8^\circ\text{C}$ (based on similar modifications)
- Evidence Chain: PETase $\rightarrow$ hydrolase family $\rightarrow$ thermostable homologs $\rightarrow$ structural alignment $\rightarrow$ mutation suggestion

11. Conclusion

QDesign represents a significant evolution in AI-assisted protein engineering, transitioning from a retrieval platform to a reasoning platform. The completion of Phase 1 validates the core data pipeline and vector-native architecture, while the ongoing development of Phases 2-4 will enable explainable design generation and collaborative workflows.

11.1. Key Innovations

- **Vector-Native Architecture:** Biological similarity as operational foundation
- **Multimodal Knowledge Integration:** Unified representation of diverse data types
- **Explainable Design:** Evidence trails showing design rationale
- **Collaborative Workbench:** Team-based engineering in shared visual space

11.2. Future Impact

The QDesign platform addresses critical bottlenecks in protein engineering by:

1. Accelerating design cycles through AI-assisted reasoning
2. Improving design quality with evidence-based suggestions
3. Enabling collaborative research across distributed teams
4. Creating reproducible workflows with documented reasoning paths

11.3. Next Steps

- Complete knowledge graph construction (Phase 2)
- Implement multi-agent hypothesis generation (Phase 3)
- Validate closed-loop learning with experimental data (Phase 4)
- Expand to additional protein engineering applications

References

1. SciAgents Paper: <https://advanced.onlinelibrary.wiley.com/doi/full/10.1002/adma.202413523>
2. Qdrant Documentation: <https://qdrant.tech/documentation/>
3. Repository: <https://github.com/Moetez-Fradi/Selecao-QDesign>
4. AlphaFold Database: <https://alphafold.ebi.ac.uk/>
5. ESM Metagenomic Atlas: <https://esmatlas.com/>
6. Sentence Transformers: <https://www.sbert.net/>